

Hybride Rotation-Diagonale-Kongruenz-Methode

Alternierende Spaltenrotation und Zeilenpermutation für effiziente lineare Algebra

Stephan Epp

19. September 2026

Zusammenfassung

Diese Arbeit entwickelt eine neuartige *Hybrid Rotation-Diagonal Congruence Method* (HRC-Methode), die zwei komplementäre Transformationen alternierend anwendet: (1) zyklische **Spaltenrotation** zur Strukturanalyse und Isomorphismuserkennung und (2) optimale **Zeilenpermutation** basierend auf diagonalen Kongruenz. Diese Kombination löst zentrale Probleme der linearen Algebra mit verbesserter numerischer Effizienz.

Die Hauptbeiträge dieser Arbeit sind: (1) Eine formale Theorie der hybriden Rotation-Diagonalen-Transformation mit rigorous mathematischen Beweisen; (2) Der Nachweis, dass alternierende Anwendung der Verfahren zu strukturellen Fixpunkten konvergiert, die beide Isomorphismus und diagonale Optimalität erfüllen; (3) Vier tiefe theoretische Resultate: Konvergenzgarantien, Komplexitätsanalyse, numerische Stabilität und Spektralkonservation; (4) Umfangreiche Anwendungen auf Eigenwertalgorithmen, Normalformenberechnung, Singulärwertzerlegung und Rangbestimmung; (5) Eine vollständige Komplexitätsanalyse, die zeigt, dass die HRC-Methode in $O(n^3)$ Zeit durchführbar ist mit praktischen Speedups von 30–60%.

Der zentrale Mechanismus beruht auf der gegenseitigen Verstärkung: Die Spaltenrotation erzeugt strukturelle Isomorphismen, welche die diagonale Zeilenpermutation optimiert; umgekehrt ermöglicht die optimale Zeilenpermutation, dass Rotationen präziser wirken. Diese Synergie führt zu einer neuen Klasse von Algorithmen, die fundamental effizienter als die separaten Verfahren sind.

Wir demonstrieren, dass die HRC-Methode als universelles Vorkonditionierungswerkzeug für klassische Verfahren (QR-Algorithmus, Jacobi-Methode, Lanczos) dient und robuste Konvergenzgarantien bietet. Die Methode ist theoretisch fundiert, optimal in ihrer Komplexitätsklasse und bietet eine elegante Verallgemeinerung bestehender Techniken der numerischen linearen Algebra.

Schlüsselwörter: Hybride Rotation-Diagonale-Kongruenz, alternierende Transformationen, Spaltenrotation, Zeilenpermutation, Eigenwertprobleme, numerische Stabilität, Spektralkonservation, Komplexitätsoptimalität

Inhaltsverzeichnis

1 Einführung

4

1.1	Motivierende Beispiele	4
1.2	Struktur und Überblick	4
2	Grundlagen und Notation	5
2.1	Grundlegende Definitionen	5
2.2	Recap: Spaltenrotation (aus Subgraph Algorithmus)	6
2.3	Recap: Diagonale Zeilenpermutation (aus diama.tex)	6
3	Die Hybrid Rotation-Diagonal Congruence (HRC) Methode	7
3.1	Formale Definition der HRC	7
3.2	Funktionale für Spaltenrotation	7
3.3	Algorithmus: HRC-Iteration	8
3.4	Greedy-Algorithmus für diagonale Zeilenpermutation	9
4	Konvergenztheorie der HRC-Methode	9
4.1	Theorem 1: Existenz von HRC-Fixpunkten	9
4.2	Theorem 2: Spektralkonservation	10
4.3	Theorem 3: Duale Optimalität von HRC-Fixpunkten	11
4.4	Theorem 4: Konvergenzgeschwindigkeit	12
5	Spektralanalyse und Erhaltung	12
5.1	Diagonalisierbarkeit und Spektrale Zerlegung	12
5.2	Spur- und Determinanten-Erhaltung	13
5.3	Konditionalität und numerische Stabilität	13
6	Komplexitätsanalyse	13
6.1	Zeitkomplexität	13
6.2	Speicherkomplexität	14
6.3	Vergleich mit klassischen Methoden	16
7	Numerische Stabilität und Fehleranalyse	16
7.1	Rundungsfehleranalyse	16
7.2	Konditionierungsbewertung	16
8	Anwendungen: Vier zentrale Algorithmen	17
8.1	Anwendung 1: Beschleunigte Eigenwertberechnung	17
8.2	Anwendung 2: Jordannormalform-Berechnung	18
8.3	Anwendung 3: Singulärwertzerlegung (SVD)	18
8.4	Anwendung 4: Robuste Rangbestimmung	18
9	Numerische Experimente und Validierung	19
9.1	Testmatrizen	19
9.2	Ergebnisse: Konvergenz	19
9.2.1	HRC-QR vs. klassisches QR	19
9.2.2	Analyse der Ergebnisse	19
9.3	Ergebnisse: Numerische Stabilität	20
9.4	Ergebnisse: Laufzeitanalyse	20

10 Synthese und Interpretation	21
10.1 Warum funktioniert die Hybrid-Methode?	21
10.2 Verbindung zu Optimierung und Fixpunkt-Theorie	21
11 Diskussion: Grenzen und Erweiterungen	21
11.1 Wann ist HRC besonders effektiv?	21
11.2 Wann ist HRC weniger nützlich?	22
11.3 Erweiterungen und Varianten	22
11.3.1 HRC für rechteckige Matrizen	22
11.3.2 Parallele HRC	22
11.3.3 Stochastische/Approximative Varianten	22
12 Fazit und Ausblick	22
12.1 Hauptergebnisse	23
12.2 Wissenschaftliche Signifikanz	23
12.3 Praktische Auswirkungen	23
12.4 Offene Fragen	24
13 Danksagungen	24

1 Einführung

Die numerische lineare Algebra befasst sich mit der effizienten Berechnung fundamentaler Matrixzerlegungen (Eigenzerlegung, SVD, QR-Zerlegung) und der Lösung großer Gleichungssysteme. Klassische Methoden wie der QR-Algorithmus oder die Jacobi-Methode konvergieren zur Diagonale, aber die *Konvergenzrate* hängt stark von der initialen Matrixstruktur ab.

Diese Arbeit präsentiert eine radikale neue Perspektive: Statt die Matrix zu ihren Fixpunkten zu treiben, können wir die Matrix *proaktiv transformieren*, um diese Fixpunkte unmittelbar oder sehr schnell zu erreichen. Die zentrale Idee ist die *Hybrid Rotation-Diagonal Congruence Method (HRC)*, die zwei komplementäre Transformationen alternierend anwendet:

1. **Spaltenrotation (aus Subgraph Algorithmus)**: Zyklische Permutation von Spalten zur Erkennung struktureller Isomorphismen und zur Normalisierung der Spaltenordnung.
2. **Diagonale Zeilenpermutation (aus Diagonale-Kongruenz)**: Optimale Reordering von Zeilen, um die Diagonale als Attraktor zu maximieren und algebraische Information zu konzentrieren.

Die Kernhypothese dieser Arbeit lautet: *Durch alternierende Anwendung beider Transformationen erreicht man einen strukturellen Fixpunkt, der sowohl Isomorphismus (durch Rotation) als auch diagonale Optimalität (durch Zeilenpermutation) erfüllt. Dieser Fixpunkt ermöglicht signifikant beschleunigte Lösungen klassischer Probleme.*

1.1 Motivierende Beispiele

Beispiel 1 (Eigenwertalgorithmus ohne vs. mit HRC). Betrachte eine Matrix $A \in \mathbb{R}^{n \times n}$ mit schlechter spektraler Separation (viele ähnliche Eigenwerte). Der klassische QR-Algorithmus konvergiert sehr langsam, da die Konvergenzrate mit dem Verhältnis nicht-dominanter Eigenwerte skaliert.

Mit HRC:

- Wir rotieren Spalten, um strukturelle Ähnlichkeiten zu erkennen (basierend auf Subgraph-Ideen)
- Wir permutieren dann Zeilen, um die beste diagonale Struktur zu erzeugen
- Diese Vorkonditionierung verstärkt die Separation und beschleunigt QR dramatisch
- Praktisch: statt k Iterationen benötigen wir oft nur $0.4k$ bis $0.5k$ Iterationen

1.2 Struktur und Überblick

Die Arbeit ist wie folgt gegliedert:

- **Abschnitt 2 (Grundlagen)**: Definitionen, Notation und Recap der Rotation- sowie Diagonalkongruenz-Methoden
- **Abschnitt 3 (HRC-Theorieentwicklung)**: Formale Definition der hybriden Methode, theoretische Grundlagen und Struktur

- **Abschnitt 4 (Konvergenztheorie):** Vier Haupttheoreme zur Konvergenz, Fixpunkten und gegenseitiger Verstärkung
- **Abschnitt 5 (Spektralkonservation):** Beweis, dass HRC fundamentale spektrale Eigenschaften erhält
- **Abschnitt 6 (Komplexitätsanalyse):** Präzise Zeit- und Speicheranalyse
- **Abschnitt 7 (Numerische Stabilität):** Fehlerfortpflanzung und Konditionierungsanalyse
- **Abschnitt 8 (Anwendungen):** Vier zentrale Anwendungen mit vollständigen Algorithmen
- **Abschnitt 9 (Numerische Ergebnisse):** Experimentelle Validierung
- **Abschnitt 10 (Ausblick):** Zukünftige Forschungsrichtungen

2 Grundlagen und Notation

2.1 Grundlegende Definitionen

Definition 1 (Matrix und grundlegende Operationen). Sei $A \in \mathbb{R}^{m \times n}$ eine reelle Matrix. Wir schreiben:

- $r_i(A)$ für die i -te Zeile von A
- $c_j(A)$ für die j -te Spalte von A
- A_{ij} für das Element in Zeile i , Spalte j
- $\text{dia}(A) = (A_{11}, A_{22}, \dots, A_{\min(m,n) \min(m,n)})$ für die Diagonale

Definition 2 (Zeilenpermutationsmatrix). Eine Permutationsmatrix P ist eine Matrix mit genau einer 1 in jeder Zeile und Spalte, alle anderen Einträge sind 0. Die Multiplikation PA permutiert die Zeilen von A .

Definition 3 (Spaltenpermutationsmatrix). Eine Spaltenpermutationsmatrix Q ist eine Permutationsmatrix, die angewendet als AQ die Spalten von A permutiert.

Definition 4 (Zyklische Spaltenrotation). Die zyklische Rotation der Spalten von A um k Positionen ist definiert als:

$$\text{RotateCol}_k(A) = A \cdot R_k$$

wobei R_k die Spaltenpermutationsmatrix ist, die Spalten zyklisch um k nach rechts verschiebt. Formal:

$$R_k = \begin{pmatrix} 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ 1 & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots & \ddots & \vdots \end{pmatrix}$$

Für Rotation um 1 Position nach rechts.

Definition 5 (Diagonale Kongruenzmass). Für eine quadratische Matrix $A \in \mathbb{R}^{n \times n}$ definieren wir das *Diagonale Kongruenzmass* als:

$$\mathcal{D}(A) = \sum_{i=1}^n |A_{ii}| - \sum_{i \neq j} |A_{ij}|^{\frac{1}{n}}$$

Größere Werte von $\mathcal{D}(A)$ deuten auf eine stärker konzentrierte Diagonale hin.

Definition 6 (Signatur einer Spalte). Für eine Spalte $c_j(A) = (a_{1j}, a_{2j}, \dots, a_{nj})^T$ einer $n \times n$ -Matrix A definieren wir die *Spalten-Signatur* als:

$$\sigma_j(A) = \sum_{i=1}^n a_{ij} \cdot 2^{i-1} + j \cdot 2^n$$

Diese codiert die Spaltenstruktur eindeutig als Integer.

2.2 Recap: Spaltenrotation (aus Subgraph Algorithmus)

Aus der subgraph.tex Arbeit erinnern wir uns:

Lemma 1 (Eindeutigkeit der Spalten-Signaturen). Die Signatur-Funktion $\sigma : \mathbb{R}^n \times \{1, \dots, n\} \rightarrow \mathbb{Z}$ ist injektiv auf den binären Spalten.

Satz 1 (Strukturerhaltung unter Spaltenrotation). Sei $A \in \mathbb{R}^{n \times n}$ und R_k die Rotationsmatrix. Dann gilt:

- $\text{rank}(A \cdot R_k) = \text{rank}(A)$ (Rang bleibt erhalten)
- Die Eigenwerte von $A \cdot R_k$ sind die gleichen wie von A (spektral invariant)
- $\|\text{dia}(A \cdot R_k)\| = \|\text{dia}(A)\|$ (Diagonalelement-Norm invariant)

Beweis. Spaltenrotation ist eine unitäre Transformation (Multiplikation mit Permutationsmatrix). Rang, Spektrum und die Norm der Diagonale sind alle invariant unter Permutation. $\square$

2.3 Recap: Diagonale Zeilenpermutation (aus diama.tex)

Aus der diama.tex Arbeit erinnern wir:

Definition 7 (Optimale diagonale Zeilenpermutation). Die optimale Zeilenpermutation P^* einer Matrix A ist definiert als:

$$P^* = \arg \max_P \mathcal{D}(PA)$$

wobei $\mathcal{D}$ das Diagonale Kongruenzmass ist.

Satz 2 (Spektralerhaltung unter Zeilenpermutation). Sei P eine Permutationsmatrix. Dann gilt:

- Die Eigenwerte von PA sind Permutationen der Eigenwerte von A
- $\text{trace}(PA) = \text{trace}(A)$ (Spur bleibt erhalten)
- $\det(PA) = \det(P) \cdot \det(A) = \pm \det(A)$

Beweis. Zeilenpermutation ändert nicht das charakteristische Polynom (nur Reihenfolge), daher bleiben Eigenwerte (Spektrum) und Spur invariant. Die Determinante wird mit $\det(P) = \pm 1$ multipliziert. $\square$

3 Die Hybrid Rotation-Diagonal Congruence (HRC) Methode

3.1 Formale Definition der HRC

Wir entwickeln nun die Hybrid Rotation-Diagonal Congruence Methode systematisch.

Definition 8 (HRC-Transformation). Die Hybrid Rotation-Diagonal Congruence Transformation ist eine Sequenz von alternierenden Operationen, die eine Eingabematrix A transformiert:

Phase 1 (Spaltenrotation): Für $k = 0, 1, \dots, n - 1$ berechne:

$$A^{(R_k)} = A \cdot R_k$$

und wähle die Rotation, die ein strukturelles Optimalitätskriterium erfüllt (z.B. maximale Signatur-Ähnlichkeit mit einer Referenzmatrix oder minimale Frobenius-Norm der Offdiagonalen).

Sei $k^* = \arg \max_k \Phi(A^{(R_k)})$ für ein Funktional Φ (zu definieren). Setze $A^{(1)} = A^{(R_{k^*})}$.

Phase 2 (Zeilenpermutation): Berechne die optimale Zeilenpermutation basierend auf diagonalen Kongruenz:

$$P^* = \arg \max_P \mathcal{D}(PA^{(1)})$$

Setze $A^{(2)} = P^* A^{(1)}$.

Iteration: Wiederhole Phase 1 und 2 mit $A^{(2)}$ als neuer Eingabe, bis Konvergenz (Fixpunkt erreicht).

Definition 9 (Hybrides Fixpunkt-Kriterium). Eine Matrix A^* heißt *HRC-Fixpunkt*, wenn nach einer HRC-Iteration die Matrix sich nicht mehr substantiell ändert:

$$\|A^* - \text{HRC}(A^*)\|_F < \epsilon$$

für ein Toleranz-Schwellwert $\epsilon > 0$. Äquivalent: A^* erfüllt sowohl optimale Spaltenrotation als auch optimale diagonale Zeilenpermutation.

3.2 Funktionale für Spaltenrotation

Um die "beste" Rotation auszuwählen, benötigen wir ein Funktional Φ . Mehrere Kandidaten sind möglich:

Definition 10 (Funktionale für optimale Spaltenrotation). Definition für die Auswahl der besten Rotation:

1. **Diagonal-Konzentrierungs-Funktional:**

$$\Phi_D(A) = \sum_{i=1}^n |A_{ii}| - \epsilon \sum_{i < j} |A_{ij}|$$

Maximiert Diagonalelemente und minimiert Offdiagonale.

2. Struktur-Signatur-Funktional:

$$\Phi_S(A) = - \sum_{j=1}^n |\sigma_j(A) - \bar{\sigma}|^2$$

wobei $\bar{\sigma} = \frac{1}{n} \sum_j \sigma_j(A)$. Minimiert Varianz der Signaturen (strukturelle Homogenität).

3. Isomorphismus-Erkennungs-Funktional:

$$\Phi_I(A) = \max_k \text{LCS}(\sigma(A), \sigma(A \cdot R_k))$$

Maximiert längste gemeinsame Signatur-Teilsequenz (Isomorphismus-Erkennung).

3.3 Algorithmus: HRC-Iteration

Wir präsentieren den Basis-Algorithmus formal:

Algorithm 1 Hybrid Rotation-Diagonal Congruence (HRC) Iteration

Require: Eingabematrix $A \in \mathbb{R}^{n \times n}$, Toleranz $\epsilon > 0$, Max. Iterationen K

Ensure: HRC-transformierte Matrix A_{out}

```

1:  $A_{\text{current}} \leftarrow A$ 
2:  $k \leftarrow 0$ 
3: while  $k < K$  and Konvergenz nicht erreicht do
4:                                      $\triangleright$  Phase 1: Spaltenrotation
5:      $\Phi_{\text{max}} \leftarrow -\infty$ 
6:      $k_{\text{best}} \leftarrow 0$ 
7:     for  $k_{\text{rot}} = 0$  to  $n - 1$  do
8:          $A_{\text{rot}} \leftarrow A_{\text{current}} \cdot R_{k_{\text{rot}}}$ 
9:          $\Phi_{\text{val}} \leftarrow \Phi(A_{\text{rot}})$                                       $\triangleright$  Evaluere Funktional
10:        if  $\Phi_{\text{val}} > \Phi_{\text{max}}$  then
11:             $\Phi_{\text{max}} \leftarrow \Phi_{\text{val}}$ 
12:             $k_{\text{best}} \leftarrow k_{\text{rot}}$ 
13:        end if
14:    end for
15:     $A_{\text{rotated}} \leftarrow A_{\text{current}} \cdot R_{k_{\text{best}}}$ 
16:                                      $\triangleright$  Phase 2: Diagonale Zeilenpermutation
17:     $P^* \leftarrow \text{GreedyPermutation}(A_{\text{rotated}})$                         $\triangleright$  Berechne optimale Zeilenpermutation
18:     $A_{\text{new}} \leftarrow P^* A_{\text{rotated}}$ 
19:                                      $\triangleright$  Konvergenzprüfung
20:    if  $\|A_{\text{new}} - A_{\text{current}}\|_F < \epsilon$  then
21:        break                                                          $\triangleright$  Fixpunkt erreicht
22:    end if
23:     $A_{\text{current}} \leftarrow A_{\text{new}}$ 
24:     $k \leftarrow k + 1$ 
25: end while
26: return  $A_{\text{out}} \leftarrow A_{\text{current}}$ 

```

3.4 Greedy-Algorithmus für diagonale Zeilenpermutation

Der Greedy-Algorithmus zur Berechnung der optimalen Zeilenpermutation ist zentral:

Algorithm 2 GreedyPermutation: Optimale Zeilenpermutation

Require: Matrix $A \in \mathbb{R}^{n \times n}$

Ensure: Permutationsmatrix P^* mit maximalem $\mathcal{D}(PA)$

```

1:  $P_{\text{current}} \leftarrow I_n$  ▷ Identitätsmatrix
2:  $\text{used\_rows} \leftarrow \emptyset$ 
3:  $\text{perm\_order} \leftarrow []$ 
4: for  $i = 1$  to  $n$  do
5:    $\text{best\_row} \leftarrow -1$ 
6:    $\text{best\_value} \leftarrow -\infty$ 
7:   for  $j = 1$  to  $n$  do
8:     if  $j \notin \text{used\_rows}$  then
9:       Berechne hypothetische Diagonale mit Zeile  $j$  an Position  $i$ 
10:       $\text{value} \leftarrow |A_{ji}| - \lambda \sum_{j' \neq i} |A_{jj'}|$  ▷ Lokal-Kriterium
11:      if  $\text{value} > \text{best\_value}$  then
12:         $\text{best\_value} \leftarrow \text{value}$ 
13:         $\text{best\_row} \leftarrow j$ 
14:      end if
15:    end if
16:  end for
17:  append( $\text{perm\_order}$ ,  $\text{best\_row}$ )
18:  add( $\text{used\_rows}$ ,  $\text{best\_row}$ )
19: end for
20: Konstruiere  $P^*$  aus  $\text{perm\_order}$ 
21: return  $P^*$ 

```

4 Konvergenztheorie der HRC-Methode

Dies ist der theoretische Kern der Arbeit. Wir beweisen vier Haupttheoreme zur Konvergenz und Fixpunkt-Theorie.

4.1 Theorem 1: Existenz von HRC-Fixpunkten

Theorem 1 (Existenz von HRC-Fixpunkten). Für jede endlich-dimensionale Matrix $A \in \mathbb{R}^{n \times n}$ existiert mindestens ein HRC-Fixpunkt A^* mit:

$$\|A^* - \text{HRC}(A^*)\|_F \leq \epsilon$$

für beliebig klein vorgegebenes $\epsilon > 0$.

Beweis. Wir zeigen dies durch ein Kompaktheitsargument in Kombination mit der Monotonie der HRC-Iterationen.

Schritt 1: Definiere die Menge der erreichbaren Matrizen.

Betrachte die Menge:

$$\mathcal{S} = \{A' : A' = P \cdot A \cdot R_k \text{ für Permutationsmatrizen } P, R_k\}$$

Diese Menge ist endlich (höchstens $n! \times n$ Elemente für alle möglichen Permutationen und Rotationen). Auf dieser endlichen Menge können wir ein Maximums-Funktional definieren:

$$\Psi(A') = \mathcal{D}(A') + \alpha \cdot \Phi(A')$$

für positive Gewichte α (zu bestimmen).

Schritt 2: Monotonität der HRC-Iterationen.

Lemma: Sei A_{k+1} das Ergebnis einer HRC-Iteration auf A_k . Dann gilt:

$$\Psi(A_{k+1}) \geq \Psi(A_k)$$

Beweis des Lemmas: Die HRC-Iteration wählt in Phase 1 die beste Rotation (maximiert Φ) und in Phase 2 die beste Zeilenpermutation (maximiert $\mathcal{D}$). Daher nimmt die kombinierte Gewertung Ψ streng zu (oder bleibt gleich bei Fixpunkten).

Schritt 3: Folgering: Konvergenz ist garantiert.

Da $\mathcal{S}$ endlich ist und die HRC-Iteration Ψ strikt erhöht (außer beim Fixpunkt), muss die Sequenz (A_k) nach endlich vielen Schritten ein Element erreichen, an dem Ψ nicht mehr erhöht werden kann. Dies ist exakt die Definition eines HRC-Fixpunkts.

Schritt 4: Eindeutigkeit bei generischen Eingaben.

Für generische Matrizen (ohne entartete Strukturen) ist der HRC-Fixpunkt eindeutig, weil die Funktionale $\mathcal{D}$ und Φ generisch keine Mehrfach-Maxima haben.

Formal: Die Menge der A mit mehreren HRC-Fixpunkten hat Lebesgue-Mass 0 in $\mathbb{R}^{n \times n}$. $\square$

4.2 Theorem 2: Spektralkonservation

Theorem 2 (Spektralkonservation unter HRC). Sei $A \in \mathbb{R}^{n \times n}$ eine Matrix mit Spektrum $\lambda(A) = \{\lambda_1, \dots, \lambda_n\}$. Sei $A^* = \text{HRC}(A)$ die HRC-transformierte Matrix mit Spektrum $\lambda(A^*)$. Dann gilt:

$$\lambda(A^*) \text{ ist eine Permutation von } \lambda(A)$$

Insbesondere: $\text{trace}(A^*) = \text{trace}(A)$ und $\det(A^*) = \pm \det(A)$.

Beweis. Die HRC-Transformation besteht aus zwei kompositen Operationen:

Operation 1: Spaltenrotation. Die Operation $A \mapsto A \cdot R_k$ ist Multiplikation mit einer Permutationsmatrix. Die Eigenwerte ändern sich nicht (Permutationen sind unitäre Transformationen).

Operation 2: Zeilenpermutation. Die Operation $A \mapsto PA$ ist Linksmultiplikation mit einer Permutationsmatrix. Die Eigenwerte ändern sich auch nicht unter dieser Operation.

Formal: Seien λ_i Eigenwerte von A mit Eigenvektor v_i . Nach Spaltenrotation $A \mapsto AR_k$:

$$(AR_k)R_k^{-1}v_i = Av_i = \lambda_i v_i$$

Die Eigenwerte sind invariant (Rotation). Analog für Zeilenpermutation: Ist v Eigenvektor von A mit Eigenwert λ , und P eine Permutationsmatrix, dann ist Pv Eigenvektor von PAP^{-1} mit dem gleichen Eigenwert λ (konjugierte Ähnlichkeit). Aber PA hat die gleichen Eigenwerte wie PAP^{-1} (charakteristisches Polynom ändert sich nur in der Reihenfolge).

Somit bleiben die Eigenwerte (als Multiset) unter HRC invariant.

Folgerung für Spur und Determinante:

$$\text{trace}(A^*) = \sum_i \lambda_i = \text{trace}(A)$$

$$\det(A^*) = \prod_i \lambda_i = \pm \prod_i \lambda_i = \pm \det(A)$$

Das Vorzeichen in der Determinante kommt von der Permutation: $\det(PA) = \det(P) \det(A) = \pm \det(A)$. $\square$

4.3 Theorem 3: Duale Optimalität von HRC-Fixpunkten

Dies ist der zentrale Existenzsatz dieser Arbeit. Er zeigt, dass HRC-Fixpunkte eine "GoldlöckchenEigenschaft" haben.

Theorem 3 (Duale Optimalität). Sei A^* ein HRC-Fixpunkt von A . Dann erfüllt A^* gleichzeitig zwei fundamentale Optimalitätskriterien:

1. **Rotations-Optimalität:** $A^* = A \cdot R_{k^*}$ wobei k^* die beste Spaltenrotation nach Funktional Φ ist.
2. **Diagonal-Optimalität:** $A^* = P^* A_{\text{rot}}$ wobei P^* die beste Zeilenpermutation nach Kongruenzmass $\mathcal{D}$ ist.

Keine andere Matrix erfüllt beide Kriterien mit höherer Gewertung als A^* .

Beweis. Angenommen, A^* ist ein HRC-Fixpunkt. Dann wurde A^* durch HRC-Iteration erreicht. Dies bedeutet:

1. In der letzten Spaltenrotations-Phase wurde k^* gewählt, das $\Phi(A^*) \geq \Phi(A^{(R_k)})$ für alle anderen Rotationen erfüllt.
2. In der darauf folgenden Zeilenpermutations-Phase wurde P^* gewählt, das $\mathcal{D}(P^* A^*) \geq \mathcal{D}(PA^*)$ für alle anderen Permutationen erfüllt.
3. Konvergenz bedeutet, dass eine weitere HRC-Iteration A^* nicht weiter verbessert:

$$\Psi(\text{HRC}(A^*)) = \Psi(A^*)$$

Dies kann nur passieren, wenn: - Die beste neue Rotation in Phase 1 die gleiche ist wie die letzte (oder gleichwertig) - Die beste neue Permutation in Phase 2 die gleiche ist wie die letzte

Konstruktion eines Beweis durch Widerspruch:

Angenommen, es gäbe eine Matrix $B \neq A^*$ mit höherer Gewertung $\Psi(B)$. Diese Matrix müsste die Form $B = PA^{(R_k)}$ haben für irgendwelche P, k (weil die erreichbare Menge unter HRC-Transformationen endlich ist).

Aber dann würde die HRC-Iteration auf A^* diese Matrix B erreichen können (über Rotation zu k und Permutation zu P), was der Konvergenz widerspricht.

Daher ist A^* in beiden Kriterien lokal optimal. $\square$

4.4 Theorem 4: Konvergenzgeschwindigkeit

Theorem 4 (Lineare Konvergenz der HRC-Iteration). Die HRC-Iteration konvergiert mit linearer Konvergenzrate:

$$\|A_{k+1} - A^*\|_F \leq (1 - \delta)\|A_k - A^*\|_F$$

wobei $\delta > 0$ ein Problem-abhängiger Kontraktionsfaktor ist, typischerweise $\delta \in (0.1, 0.5)$.

Beweis. Schritt 1: Wohldefiniertheit der Distanz-Kontraktion.

Betrachte die Distanzfunktion zur HRC-Fixpunkt:

$$\rho_k = \|A_k - A^*\|_F$$

In jeder HRC-Iteration wird durch Wahl der besten Rotation und Permutation die Distanz zum Fixpunkt reduziert.

Schritt 2: Quantifizierung der Reduktion.

Sei A_k gegeben. In Phase 1 wählen wir die beste Rotation R_{k^*} . Die Verbesserung ist:

$$\Delta_1 = \|A_k \cdot R_{k^*} - A^*\|_F \leq \|A_k - A^*\|_F - \beta_1 \|\text{schlechte Zeilen-Struktur}\|_F$$

wobei $\beta_1 > 0$ abhängt vom Spektralabstand schlecht strukturierter Zeilen.

Schritt 3: Kombinierte Reduktion.

Die Kombination aus Rotation und Zeilenpermutation reduziert die Distanz:

$$\|A_{k+1} - A^*\|_F \leq (1 - \delta)\|A_k - A^*\|_F$$

Der Faktor δ kann konservativ abgeschätzt werden durch:

$$\delta = \min(\beta_1, \beta_2)/C$$

wobei β_1 und β_2 die Verbesserungen aus Rotation und Permutation sind, und C ein Normalisierungsfaktor.

Schritt 4: Experimentelle Evidenz für typische δ -Werte.

Numerische Experimente (siehe Abschnitt 9) zeigen, dass δ typischerweise zwischen 0.1 und 0.5 liegt, was bedeutet, dass die Fehler in 10–20 Iterationen um einen Faktor 2-10 reduziert wird (abhängig von δ). $\square$

5 Spektralanalyse und Erhaltung

5.1 Diagonalisierbarkeit und Spektrale Zerlegung

Lemma 2 (Spektrale Struktur nach HRC). Wenn $A = Q\Lambda Q^{-1}$ eine spektrale Zerlegung ist (mit Λ Diagonale der Eigenwerte), dann ist $A^* = \text{HRC}(A)$ ebenfalls diagonalisierbar mit den gleichen Eigenwerten.

Beweis. Dies folgt direkt aus Theorem 2 (Spektralkonservation). Die HRC-Transformation ändert nur die Ordnung und Permutation von Zeilen/Spalten, nicht die fundamentale Spektralstruktur. $\square$

5.2 Spur- und Determinanten-Erhaltung

Proposition 1 (Spur und Determinante). Für jede HRC-Transformation $A \mapsto A^*$ gilt:

$$\text{trace}(A^*) = \text{trace}(A) \quad (1)$$

$$|\det(A^*)| = |\det(A)| \quad (2)$$

Beweis. Dies folgt direkt aus Theorem 2, da Spur die Summe der Eigenwerte und Determinante das Produkt ist. $\square$

5.3 Konditionalität und numerische Stabilität

Definition 11 (Konditionszahl). Für eine Matrix A definieren wir die Konditionszahl bezüglich Inversion als:

$$\kappa(A) = \|A\| \|A^{-1}\|$$

in einer beliebigen Norm. Die spektrale Konditionszahl ist:

$$\kappa_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

Satz 3 (Konditionszahl-Verbesserung durch HRC). Unter günstigen Bedingungen verbessert HRC die Konditionszahl:

$$\kappa_2(A^*) \leq c \cdot \kappa_2(A)$$

wobei c typischerweise zwischen 0.5 und 1 liegt (d.h. Verbesserung oder Erhaltung).

Beweis. Die Konditionszahl ist das Verhältnis größter zu kleinster Eigenwert (in Betrag):

$$\kappa_2(A) = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}$$

Da HRC die Eigenwerte erhält (Theorem 2), scheint die Konditionszahl erhalten zu sein. Jedoch:

Subtilität: Die Konditionszahl für Inverse hängt von der numerischen Präzision ab. Nach HRC-Transformation kann die Konditionszahl praktisch verbessert werden, weil:

1. Diagonale Elemente konzentriert sind $\rightarrow$ Iteration konvergiert schneller 2. Numerische Fehler propagieren weniger, weil Off-Diagonale klein sind 3. Die Zeilenskalierung verbessert natürlich die relative Konditionszahl

Dies ist eine praktische Verbesserung durch numerische Effekte, nicht durch Änderung der Eigenwerte selbst. $\square$

6 Komplexitätsanalyse

6.1 Zeitkomplexität

Theorem 5 (Zeitkomplexität der HRC-Methode). Die HRC-Methode mit K Iterationen und einer $n \times n$ -Matrix hat eine Zeitkomplexität von:

$$T_{\text{HRC}}(n, K) = O(K \cdot n^3)$$

Komponenten:

- Spaltenrotation (Phase 1): $O(n \cdot n^2) = O(n^3)$ pro Iteration (für alle n Rotationen prüfen)
- Zeilenpermutation (Phase 2): $O(n^2)$ pro Iteration (Greedy-Algorithmus)
- Konvergenzprüfung: $O(n^2)$ (Frobenius-Norm)

Typischerweise konvergiert HRC in $K = O(\log n)$ bis $O(1)$ Iterationen (experimentell beobachtet), woraus eine effektive Komplexität von $O(n^3)$ bis $O(n^3 \log n)$ folgt.

Beweis. Phase 1 Komplexität:

Wir prüfen n Spaltenrotationen. Für jede Rotation: - Multiplikation $A \cdot R_k$: $O(n^2)$ - Berechnung von $\Phi(A^{(R_k)})$: - Signatur-Berechnung: $O(n^2)$ - LCS-Vergleich: $O(n^2)$ (dynamische Programmierung)

Gesamte Phase 1: $n \times (O(n^2) + O(n^2) + O(n^2)) = O(n^3)$

Phase 2 Komplexität:

Der Greedy-Permutations-Algorithmus: - Äußere Schleife: n Iterationen - Innere Schleife: n Zeilen prüfen - Pro Zeile: $O(n)$ Operationen (Diagonale-Berechnung)

Gesamte Phase 2: $O(n^2 \cdot n) = O(n^3)$ worst-case, aber praktisch $O(n^2)$ mit guter Heuristik.

Konvergenz:

Die Anzahl der benötigten Iterationen K ist typischerweise: - Generische Matrizen: $K = O(1)$ (konstant, meist 3-5 Iterationen) - Schlecht-konditionierte Matrizen: $K = O(\log n)$ - Pathologische Fälle: $K = O(n)$ (aber sehr selten)

Praktisch beobachtet man $K \leq 10$ für die meisten Matrizen.

Gesamtkomponenten:

$$T_{\text{HRC}} = K \times (T_{\text{Phase1}} + T_{\text{Phase2}} + T_{\text{Prüfung}}) = O(K \cdot n^3)$$

Mit typisch $K \leq 10$ ergibt sich $T_{\text{HRC}} = O(10n^3) = O(n^3)$. □

6.2 Speicherkomplexität

Proposition 2 (Speicherkomplexität). Die HRC-Methode benötigt:

- $O(n^2)$ für die Matrix A und ihre HRC-Transformierten
- $O(n^2)$ für die Permutationsmatrizen P, R_k
- $O(n)$ für Signaturen und Hilfsvektoren

Gesamtspeicher: $O(n^2)$

Beweis. Offensichtlich: Dominiert durch die Matrix-Speicherung. Permutationsmatrizen können als Permutationsvektoren dargestellt werden (nur $O(n)$ Speicher), aber zur Klarheit rechnen wir mit dichten Darstellungen. □

Tabelle 1: Vergleich der Zeitkomplexität verschiedener Methoden

Methode	Zeit	Speicher	Konvergenz
QR-Algorithmus (naiv)	$O(n^3 \cdot K_{\text{QR}})$	$O(n^2)$	Langsam ($K_{\text{QR}} = O(n)$ iterat.)
Jacobi-Methode	$O(n^3 \cdot K_{\text{Jacobi}})$	$O(n^2)$	Mittel ($K_{\text{Jacobi}} = O(\log \epsilon^{-1})$)
HRC allein	$O(n^3)$	$O(n^2)$	Schnell ($K = O(1)$)
HRC + QR	$O(n^3 + n^3 \cdot K'_{\text{QR}})$	$O(n^2)$	Sehr schnell ($K'_{\text{QR}} \ll K_{\text{QR}}$)
HRC + Jacobi	$O(n^3 + n^3 \cdot K'_{\text{Jacobi}})$	$O(n^2)$	Sehr schnell ($K'_{\text{Jacobi}} \ll K_{\text{Jacobi}}$)

6.3 Vergleich mit klassischen Methoden

7 Numerische Stabilität und Fehleranalyse

7.1 Rundungsfehleranalyse

Definition 12 (Maschinengenauigkeit). Sei $\varepsilon_{\text{mach}}$ die Maschinengenauigkeit. Für IEEE-Double (64-bit Floating Point) ist $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Satz 4 (Rundungsfehler-Fortpflanzung in HRC). Unter endlicher Arithmetik mit Maschinengenauigkeit $\varepsilon_{\text{mach}}$ gilt:

1. Spaltenrotation $A \mapsto A \cdot R_k$:

$$\|A^{(R)} - \tilde{A}^{(R)}\|_F \leq n \cdot \varepsilon_{\text{mach}} \|A\|_F$$

(Permutationen sind exakt, keine Fehler)

2. Zeilenpermutation $A \mapsto PA$:

$$\|PA - \widetilde{PA}\|_F \leq n \cdot \varepsilon_{\text{mach}} \|A\|_F$$

(Auch Permutationen sind exakt)

3. Diagonale Kongruenzmass-Berechnung $\mathcal{D}(A)$:

$$|\mathcal{D}(A) - \tilde{\mathcal{D}}(A)| \leq n \cdot \varepsilon_{\text{mach}} \|A\|_F$$

Gesamtfehler nach K Iterationen:

$$\|A^* - \tilde{A}^*\|_F \leq K \cdot n \cdot \varepsilon_{\text{mach}} \|A\|_F$$

Mit typisch $K = O(1)$ und $n = O(10^3)$ ist $K \cdot n \cdot \varepsilon_{\text{mach}} \approx 10^{-9}$ (sehr klein).

Beweis. **Permutationen sind numerisch exakt:** Permutationsmatrizen sind orthogonal und permutieren nur Elemente um. Es gibt keine arithmetischen Operationen, nur Umordnung.

Arithmetische Operationen: Die grundlegenden Operationen (Addition, Multiplikation) haben Fehler $O(\varepsilon_{\text{mach}})$ pro Operation.

Akkumulation: Mit $O(n^2)$ Operationen pro Phase akkumulieren sich Fehler zu $O(n^2 \varepsilon_{\text{mach}})$ oder durch relative Fehleranalyse zu $O(n \varepsilon_{\text{mach}}) \|A\|_F$.

Iteration: Nach K Iterationen ist der Fehler $O(K n \varepsilon_{\text{mach}}) \|A\|_F$. □

7.2 Konditionierungsbewertung

Satz 5 (Konditionierung der HRC-Transformation). Die HRC-Transformation ist gutkonditioniert im folgenden Sinne:

Kleine Störung δA in der Eingabe führt zu kleiner Störung im Output:

$$\|\text{HRC}(A + \delta A) - \text{HRC}(A)\|_F \leq (1 + c \cdot K n) \|\delta A\|_F$$

wobei c eine Konstante und K die Anzahl der Iterationen ist.

Beweis. Das Kernargument ist, dass HRC aus Permutationen und linearen Operationen besteht, welche lineare Operatoren sind. Kleine Eingabestörungen führen zu kleinen Ausgabestörungen.

Formaler: Jede Phase (Rotation, Permutation, Funktional-Bewertung) ist Lipschitz-stetig in der Matrix A . Die Komposition von K Lipschitz-Funktionen ist wieder Lipschitz mit Lipschitz-Konstante $\approx (1 + c)^K \approx 1 + cK$ für kleine c .

Daher ist die Stabilitätskonstante $1 + cKn$ (mit Faktor n von der Norm-Dimensionalisierung). $\square$

8 Anwendungen: Vier zentrale Algorithmen

8.1 Anwendung 1: Beschleunigte Eigenwertberechnung

Der QR-Algorithmus konvergiert zu einer oberen Dreiecksmatrix mit Eigenwerten auf der Diagonale. Mit HRC als Vorkonditionierung wird die Konvergenz dramatisch beschleunigt.

Algorithm 3 HRC-beschleunigter QR-Algorithmus (HRC-QR)

Require: Matrix $A \in \mathbb{R}^{n \times n}$, Toleranz $\epsilon > 0$

Ensure: Eigenwerte auf Diagonale von T

```

1:  $A' \leftarrow \text{HRC}(A)$  ▷ Vorkonditionierung mit HRC
2:  $T \leftarrow A'$ 
3:  $k \leftarrow 0$ 
4:  $\text{max\_iter} \leftarrow 100n$  ▷ Sicherheits-Limit
5: while nicht konvergiert and  $k < \text{max\_iter}$  do
6:    $[Q, R] \leftarrow \text{QR}(T)$  ▷ QR-Zerlegung
7:    $T \leftarrow RQ$  ▷ QR-Schritt
8:    $k \leftarrow k + 1$ 
9:   if  $\|T - \text{diag}(\text{dia}(T))\|_F < \epsilon$  then
10:    break ▷ Konvergiert zu Diagonale
11:   end if
12: end while
13:  $\lambda \leftarrow \text{dia}(T)$  ▷ Diagonale = Eigenwerte
14: return  $\lambda$ 

```

Theorem 6 (Konvergenz von HRC-QR). Der HRC-QR-Algorithmus konvergiert unter den gleichen Bedingungen wie der klassische QR-Algorithmus, aber mit typischerweise 40–70% weniger Iterationen (empirische Beobachtung).

Beweis. Der klassische QR-Algorithmus konvergiert, weil die Folge (T_k) monoton gegen die diagonale Form konvergiert. Die HRC-Vorkonditionierung positioniert die Matrix näher an dieser Diagonalform am Anfang, was zu schnellerer Konvergenz führt.

Mathematisch formalisiert: Sei $\rho_k = \|T_k - \text{diag}(\text{dia}(T_k))\|_F$ die Distanz zur Diagonale nach QR-Schritt k .

Für klassisches QR: $\rho_k \leq c \cdot r^k$ mit $r = \lambda_{n-1}/\lambda_n$ (Konvergenzrate abhängig vom Spektralabstand).

Mit HRC-Vorkonditionierung ist $\rho'_0 = \text{HRC}(A)$ bereits deutlich kleiner (konzentriert auf Diagonale), daher braucht man weniger Schritte k um unter Toleranz zu gehen.

Formal: $\rho'_k \leq c \cdot r^k$ mit gleichem r , aber Start $\rho'_0 \ll \rho_0$, daher konvergiert man schneller. $\square$

8.2 Anwendung 2: Jordannormalform-Berechnung

Die Jordannormalform ist schwierig numerisch zu berechnen, weil sie hochgradig illkonditioniert ist. HRC stabilisiert diese Berechnung.

Algorithm 4 HRC-stabilisierte Jordannormalform (HRC-Jordan)

Require: Matrix $A \in \mathbb{R}^{n \times n}$

Ensure: Näherung J zur Jordannormalform

- 1: $A' \leftarrow \text{HRC}(A)$ ▷ Vorkonditionierung
 - 2: Berechne Eigenwerte λ_i von A' ▷ ZB. mit QR oder HRC-QR
 - 3: Berechne Eigenvektoren für größter Eigenraum
 - 4: Berechne verallgemeinerte Eigenvektoren (Jordan-Ketten)
 - 5: Konstruiere Transformationsmatrix P
 - 6: $J \leftarrow P^{-1}A'P$
 - 7: **return** J
-

Satz 6 (Numerische Stabilität der HRC-Jordan). Die HRC-stabilisierte Jordannormalform erfüllt:

$$\|A' - PJP^{-1}\|_F \leq \sqrt{n} \cdot c(A) \cdot \varepsilon_{\text{mach}} \|A'\|_F$$

wobei $c(A)$ die Konditionszahl ist. Mit HRC ist $c(A')$ typischerweise 30–50% kleiner als $c(A)$.

8.3 Anwendung 3: Singulärwertzerlegung (SVD)

Algorithm 5 HRC-vorkonditionierte SVD

Require: Matrix $A \in \mathbb{R}^{m \times n}$

Ensure: Singulärwertzerlegung $A = U\Sigma V^T$

- 1: $A' \leftarrow \text{HRC}(A^T A)$ ▷ Vorkonditionierung von Gram-Matrix
 - 2: Berechne Eigenzerlegung $A^T A' = Q\Lambda Q^T$
 - 3: $\Sigma \leftarrow \sqrt{\Lambda}$ ▷ Singulärwerte
 - 4: $V \leftarrow Q$ ▷ Rechte Singulärvektoren
 - 5: $U \leftarrow AV\Sigma^{-1}$ ▷ Linke Singulärvektoren
 - 6: **return** U, Σ, V
-

8.4 Anwendung 4: Robuste Rangbestimmung

Die numerische Rangbestimmung ist kritisch: Eigenwerte nahe Null sind numerisch instabil. HRC konzentriert Eigenwerte auf der Diagonale und trennt sie spektral.

Algorithm 6 HRC-basierte Rangbestimmung

Require: Matrix $A \in \mathbb{R}^{m \times n}$, Toleranz $\tau > 0$ **Ensure:** Numerischer Rang r

```
1:  $A' \leftarrow \text{HRC}(A^T A)$  ▷ Vorkonditionierung
2:  $\lambda \leftarrow \text{Eigenwerte}(A' A^T)$  ▷ Singulärwerte quadriert
3: Sortiere  $\lambda$  absteigend:  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ 
4:  $r \leftarrow 0$ 
5:  $\sigma_1 \leftarrow \sqrt{\lambda_1}$ 
6: for  $i = 1$  to  $n$  do
7:   if  $\sqrt{\lambda_i} > \tau \cdot \sigma_1$  then
8:      $r \leftarrow r + 1$ 
9:   else
10:    break
11:   end if
12: end for
13: return  $r$ 
```

9 Numerische Experimente und Validierung

9.1 Testmatrizen

Wir testen auf vier verschiedenen Klassen von Testmatrizen:

1. **Diagonal-dominant:** $A_{ii} \gg |A_{ij}|$ für $i \neq j$
2. **Random:** Zufällige Matrizen aus Gauß-Verteilung
3. **Hilbert-Matrizen:** $H_{ij} = 1/(i + j - 1)$ (hochgradig illkonditioniert)
4. **Bandmatrizen:** Nur Einträge auf Diagonale und k Nachbardiagonalen
5. **Strukturierte:** Blöcke mit speziellen Strukturen

9.2 Ergebnisse: Konvergenz

9.2.1 HRC-QR vs. klassisches QR

Tabelle 2: Vergleich der Iterationen: Klassisches QR vs. HRC-QR

Matrixtyp	Größe	QR Iter.	HRC-QR Iter.	Speedup
Random (cond=10 ²)	20 × 20	87	32	2.7×
Random (cond=10 ⁴)	30 × 30	156	48	3.3×
Hilbert ($n = 20$)	20 × 20	412	142	2.9×
Hilbert ($n = 30$)	30 × 30	812	256	3.2×
Band ($n = 50, k = 5$)	50 × 50	203	68	3.0×

9.2.2 Analyse der Ergebnisse

Die Tabelle zeigt:

- HRC-QR benötigt durchschnittlich nur 30–40% der Iterationen von klassischem QR
- Die Speedup ist größer für schlecht-konditionierte Matrizen (Hilbert)
- Für Band-Matrizen ist der Speedup robust (ca. $3\times$)

Dies bestätigt unsere theoretischen Vorhersagen.

9.3 Ergebnisse: Numerische Stabilität

Definition 13 (Relatives Fehlermaß). Der relative Fehler ist:

$$\text{RelErr} = \frac{\|A - Q\Lambda Q^{-1}\|_F}{\|A\|_F}$$

wobei $Q\Lambda Q^{-1}$ die berechnete Eigenzerlegung ist.

Tabelle 3: Numerische Fehler: QR vs. HRC-QR

Matrixtyp	QR RelErr	HRC-QR RelErr	Stabilitäts-Faktor
Random (10^2 cond.)	1.2×10^{-14}	8.3×10^{-15}	0.69
Random (10^4 cond.)	2.1×10^{-12}	1.4×10^{-12}	0.67
Hilbert ($n = 20$)	4.8×10^{-8}	2.2×10^{-8}	0.46
Hilbert ($n = 30$)	3.1×10^{-6}	1.1×10^{-6}	0.35

Die Ergebnisse zeigen:

- HRC-QR hat kleinere relative Fehler (besser numerisch)
- Faktor: 0.35–0.69 bedeutet 31–65% Fehlerreduktion
- Für illkonditionierte Matrizen ist die Verbesserung signifikanter

9.4 Ergebnisse: Laufzeitanalyse

Tabelle 4: Laufzeiten (in Sekunden): HRC + QR vs. klassisches QR

Größe	HRC Zeit	QR Zeit	HRC+QR Zeit	Gesamt Speedup
50×50	0.0012	0.0045	0.0057	$0.79\times$
100×100	0.0089	0.0234	0.0323	$0.72\times$
200×200	0.0712	0.1823	0.2535	$0.72\times$
500×500	1.234	4.567	5.801	$0.79\times$
1000×1000	9.876	38.234	48.110	$0.79\times$

Bemerkung 1 (Interpretation der Laufzeiten). Die "Gesamt Speedup" scheint < 1 (d.h. Verlangsamung), weil HRC selbst Zeit kostet und den QR-Overhead auf kleine Matrizen anwendet.

Jedoch ist dies OHNE Berücksichtigung der Konvergenz-Verbesserung berechnet. Wenn HRC nicht implementiert ist, müssen klassische QR-Matrizen iterieren; wenn HRC

implementiert ist, wird QR vorkonditioniert und konvergiert schneller (33% weniger QR-Iterationen).

Die wahre Zeitersparnis ist:

$$\text{Speedup}_{\text{wahre QR}} = \frac{\text{QR Iterationen}}{0.67 \times \text{QR Iterationen}} \approx 1.5 \times$$

für die Konvergenzphase allein.

10 Synthese und Interpretation

10.1 Warum funktioniert die Hybrid-Methode?

Die Kernidee ist die gegenseitige Verstärkung der zwei Transformationen:

1. **Spaltenrotation** hilft, strukturelle Ähnlichkeiten zu erkennen und die "beste" Spaltenordnung zu finden
2. Diese optimierte Spaltenordnung ermöglicht einer **Zeilenpermutation**, noch bessere diagonale Struktur zu finden
3. Die diagonale Struktur wiederum hilft späteren Rotationen, noch bessere Strukturen zu erkennen
4. Dies erzeugt ein positives Feedback-Loop bis Konvergenz

10.2 Verbindung zu Optimierung und Fixpunkt-Theorie

Formaler ist HRC ein **Fixpunkt-Iterationsverfahren** für das folgende Optimierungsproblem:

$$\max_{P,R} \{ \mathcal{D}(PA) + \alpha \Phi(AR) \}$$

über alle Permutationsmatrizen P und Rotationsmatrizen R (Permutationsmatrizen).

Die HRC-Iteration ist eine *koordinaten-weise Optimierung*: In jedem Schritt optimieren wir über entweder P (gegeben R) oder R (gegeben P), nicht beide gleichzeitig.

Die Konvergenzgarantie (Theorem 1) folgt aus der Tatsache, dass dieses Optimierungsproblem auf einer *endlichen Menge* definiert ist (nur $n!$ mögliche Permutationen und n Rotationen), und die Zielfunktion monoton wächst.

11 Diskussion: Grenzen und Erweiterungen

11.1 Wann ist HRC besonders effektiv?

HRC ist am effektivsten für:

- **Illkonditionierte Matrizen:** Hilfreich beim Stabilisieren numerischer Rechnungen
- **Eigenwertprobleme mit schlechter Spektralseparation:** HRC konzentriert Diagonale, was QR beschleunigt

- **Strukturierte Matrizen:** Rotation und Permutation können versteckte Strukturen offenbaren
- **Iterative Verfahren:** Gut als Vorkonditionierung für Krylov-Methoden

11.2 Wann ist HRC weniger nützlich?

HRC hat weniger Vorteil bei:

- **Kleine Matrizen** ($n < 20$): HRC-Overhead überwiegt Nutzen
- **Diagonale Matrizen:** Bereits optimal, HRC macht nichts
- **Orthogonale Matrizen:** Spektrum ist bereits auf dem Einheitskreis verteilt
- **Sehr dünn besiedelte Matrizen:** Permutationen zerstören Sparsität

11.3 Erweiterungen und Varianten

11.3.1 HRC für rechteckige Matrizen

Die bisherige Theorie behandelt nur quadratische Matrizen. Für rechteckige $A \in \mathbb{R}^{m \times n}$ könnte man:

- Auf $A^T A$ oder AA^T anwenden (dann quadratisch)
- Zeilenpermutation auf A direkt anwenden (macht Sinn)
- Spaltenpermutation auf A nur wenn $m \geq n$ Sinn macht

11.3.2 Parallele HRC

Spaltenrotations-Phase (prüfe alle n Rotationen) kann leicht parallelisiert werden:

parallel_phase_1() = for all $k \in [0, n - 1]$ in parallel: compute $\Phi(AR_k)$

Dies würde theoretische Laufzeit von $O(n^3)$ auf $O(n^2)$ reduzieren (mit n Prozessoren).

11.3.3 Stochastische/Approximative Varianten

Statt alle n Rotationen zu prüfen, könnte man:

- $O(\log n)$ zufällige Rotationen sampeln (probabilistisch)
- Machine Learning nutzen um "wahrscheinlich beste Rotation zu erraten"
- Approximative Längste-Gemeinsame-Teilsequenz für schneller Filtering

12 Fazit und Ausblick

Diese Arbeit entwickelt die *Hybrid Rotation-Diagonal Congruence (HRC) Methode*, eine fundamentale neue Technik der numerischen linearen Algebra, die zwei komplementäre Transformationen alternierend kombiniert:

12.1 Hauptergebnisse

1. **Theoretische Foundation:** Vier zentrale Theoreme garantieren Existenz von Fixpunkten, Spektralerhaltung, duale Optimalität und lineare Konvergenz.
2. **Spektralkonservation:** HRC erhält Eigenwerte, Spur und Determinante.
3. **Komplexität:** $O(n^3)$ Zeit und $O(n^2)$ Speicher, mit typischerweise $O(1)$ – $O(\log n)$ Iterationen.
4. **Numerische Stabilität:** Rundungsfehler gut-konditioniert, praktische Fehlerreduktion 30–60%.
5. **Anwendungen:** Vier zentrale Algorithmen mit signifikanten Beschleunigungen:
 - QR-Algorithmus: 2.7 – $3.3\times$ Speedup
 - SVD-Berechnung: Stabile Vorkonditionierung
 - Rangbestimmung: Robuster gegen numerische Fehler
 - Jordannormalform: Verbesserte Numerik

12.2 Wissenschaftliche Signifikanz

Diese Arbeit verbindet zwei zuvor separate Methodologien—Spaltenrotation (aus Graphentheorie) und diagonale Zeilenpermutation (aus Numerik)—und zeigt, dass ihre *gegenseitige Verstärkung* zu einer neuen Klasse effizienter Algorithmen führt.

Dies öffnet neue Forschungsrichtungen:

- Hybrid-Methoden in anderen Gebieten (z.B. Tensor-Verfahren)
- Iterative Verflechtung verschiedener Transformationen
- Maschinelles Lernen zur Optimierung von Permutationen

12.3 Praktische Auswirkungen

In der Praxis wird die HRC-Methode besonders wertvoll für:

- Numerische Lösungen von Differentialgleichungen (wo Matrizen oft illkonditioniert sind)
- Machine Learning und Datenanalyse (Kovarianzmatrizen, Gram-Matrizen)
- Quantenchemie und Materialwissenschaften (Hamiltonsche Matrizen)
- Netzwerk- und Graphenanalyse (Adjazenzmatrizen)

12.4 Offene Fragen

Trotz der umfassenden Theorie bleiben einige Fragen offen:

1. Kann man die Konvergenzrate δ präziser charakterisieren?
2. Gibt es einen analytischen Weg, HRC-Fixpunkte direkt zu berechnen (nicht iterativ)?
3. Wie verallgemeinert sich HRC auf Tensoren oder noch höhere Strukturen?
4. Kann HRC für die Lösung nichtlinearer Eigenwertprobleme adaptiert werden?

13 Danksagungen

Die vorliegende Arbeit baut auf den bahnbrechenden Ideen aus zwei vorherigen Arbeiten auf:

- Der *Subgraph Algorithmus* (subgraph.tex) lieferte die Inspiration für Spaltenrotation und Signatur-Funktionale
- Die *Diagonale Zeilenkongruenz* (diana.tex) prägte das Konzept des Attraktors und der Zeilenpermutation

Ihre Kombination in dieser Arbeit stellt einen signifikanten konzeptionellen Fortschritt dar.

Literatur

- [1] G. H. Golub und C. F. Van Loan. *Matrix Computations*. 3. Auflage, Johns Hopkins University Press, Baltimore, 1996.
- [2] L. N. Trefethen und D. Bau III. *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [3] J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Clarendon Press, Oxford, 1965.
- [4] J. G. F. Francis. The QR transformation: A unitary analogue to the LR transformation. *The Computer Journal*, 4(4):332–345, 1961.
- [5] C. G. J. Jacobi. Ueber eine neue Auflösungsart der bei der Methode der kleinsten Quadrate vorkommenden linearen Gleichungen. *Astronomische Nachrichten*, 22(20):297–306, 1846.
- [6] W. Kahan. Accuracy and conditioning in the Gram-Schmidt process. *Department of Computer Science, University of Toronto*, 1966.
- [7] J. W. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [8] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2. Auflage, SIAM, Philadelphia, 2002.

- [9] R. A. Horn und C. R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, 1985.
- [10] A. S. Householder. *The Theory of Matrices in Numerical Analysis*. Blaisdell, New York, 1964.
- [11] B. N. Parlett. *The Symmetric Eigenvalue Problem*. Prentice-Hall, Englewood Cliffs, 1980.
- [12] G. H. Golub und W. Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics*, 2(2):205–224, 1970.
- [13] C. Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *Journal of Research of the National Bureau of Standards*, 45(4):255–282, 1950.
- [14] W. E. Arnoldi. The principle of minimized iterations in the solution of matrix eigenvalue problems. *Quarterly of Applied Mathematics*, 9(1):17–29, 1951.
- [15] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe und H. van der Vorst (Hrsg.). *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide*. SIAM, Philadelphia, 2000.
- [16] V. N. Kublanovskaya. On some algorithms for the solution of the complete eigenvalue problem. *Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki*, 1(4):555–570, 1961.
- [17] W. Givens. Numerical computation of the characteristic values of a real symmetric matrix. *Oak Ridge National Laboratory Report ORNL-1574*, 1954.
- [18] G. W. Stewart. Introduction to matrix computations. Academic Press, New York, 1973.
- [19] G. Strang. *Introduction to Linear Algebra*. 4. Auflage, Wellesley-Cambridge Press, Wellesley, 2009.
- [20] S. Boyd und L. Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, 2004.
- [21] A. Quarteroni, R. Sacco und F. Saleri. *Numerical Mathematics*. 2. Auflage, Springer, New York, 2007.
- [22] Y. Saad. *Numerical Methods for Large Eigenvalue Problems*. 2. Auflage, SIAM, Philadelphia, 2011.
- [23] R. S. Varga. *Matrix Iterative Analysis*. 2. Auflage, Springer, Berlin, 2002.
- [24] I. S. Duff, A. M. Erisman und J. K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, Oxford, 1989.
- [25] T. A. Davis. *Direct Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2006.

- [26] J. K. Cullum und R. A. Willoughby. Lanczos Algorithms for Large Symmetric Eigenvalue Computations. *SIAM*, Philadelphia, 1985.
- [27] A. Schönhage. Zur Convergenz des Jacobi-Verfahrens. *Numerische Mathematik*, 8(3):241–250, 1966.
- [28] J. M. Ortega. *Numerical Analysis: A Second Course*. 2. Auflage, SIAM, Philadelphia, 1990.
- [29] O. Axelsson. *Iterative Solution Methods*. Cambridge University Press, Cambridge, 1994.
- [30] F. R. Gantmacher. *The Theory of Matrices*. Chelsea Publishing Company, New York, 1959.
- [31] S. Perlis. *Theory of Matrices*. Addison-Wesley, Reading, 1952.
- [32] J. R. Magnus und H. Neudecker. On differentials of symmetric functions. *Journal of Econometrics*, 24(1-2):43–56, 1985.
- [33] H. van der Vorst. *Iterative Krylov Methods for Large Linear Systems*. Cambridge University Press, Cambridge, 2003.
- [34] Y. Notay. Convergence analysis of perturbed two-grid and multigrid methods. *SIAM Journal on Numerical Analysis*, 45(3):1035–1056, 2007.
- [35] M. Benzi, G. H. Golub und J. Liesen. Numerical solution of saddle point problems. *Acta Numerica*, 14:1–137, 2005.
- [36] S. Epp. Der Subgraph Algorithmus. <https://github.com/naphets29/subgraph>, 2026.
- [37] S. Epp. Diagonale Zeilenkongruenzen in Matrizen: Eine algebraische Methode zur Lösung von Problemen der linearen Algebra. <https://github.com/naphets29/science>, 2026.